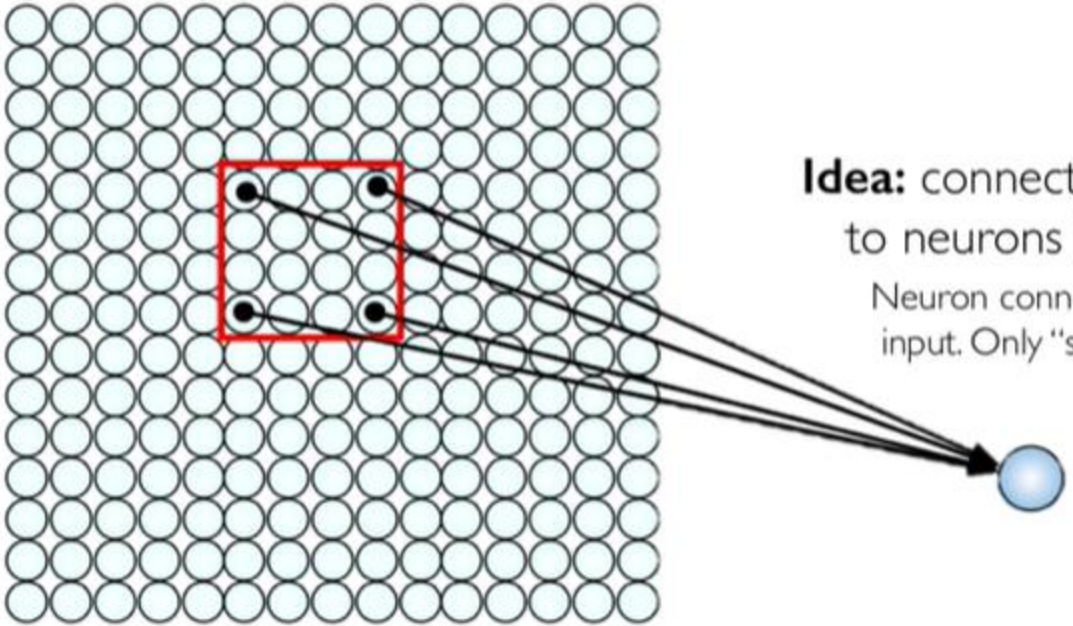
Convolutional Neural Network

Using Spatial Structure

Input: 2D image.
Array of pixel values

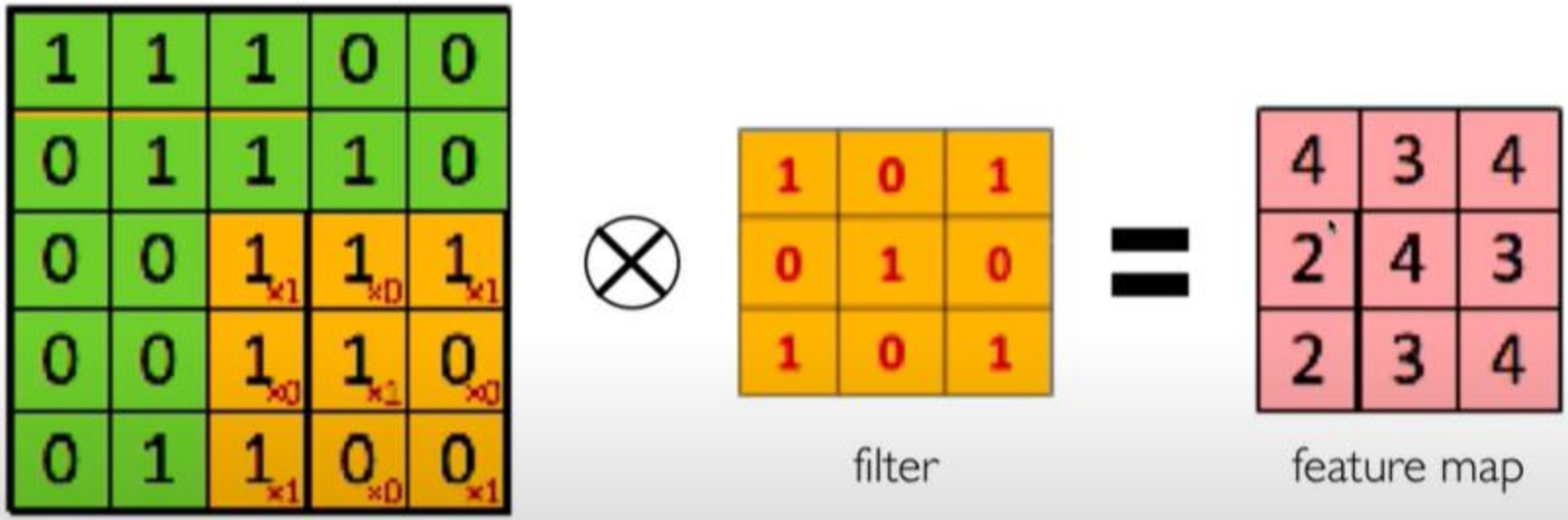


Idea: connect patches of input to neurons in hidden layer.

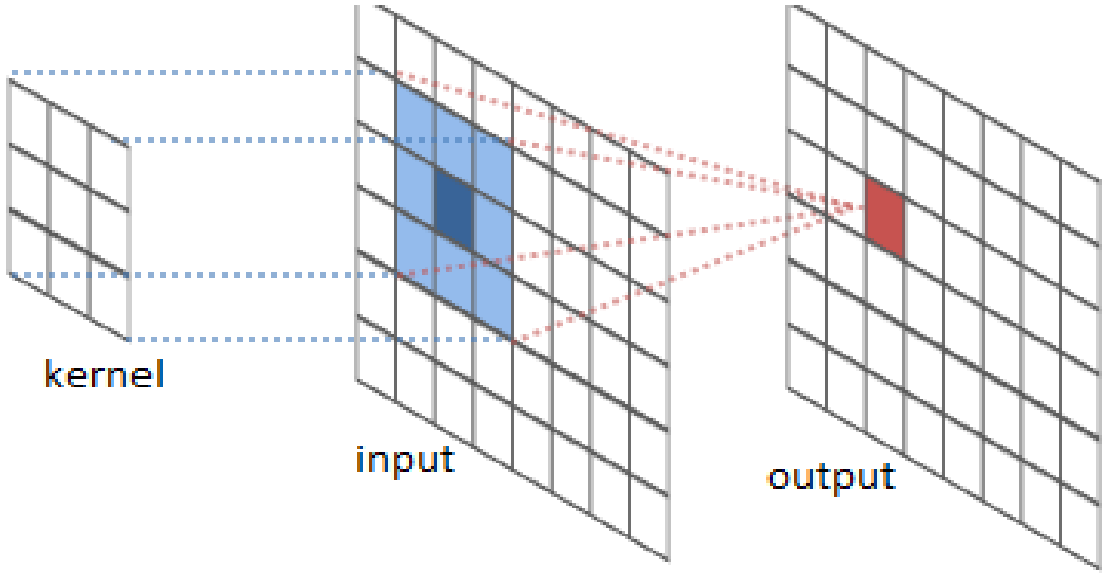
Neuron connected to region of input. Only "sees" these values.

The Convolution Operation

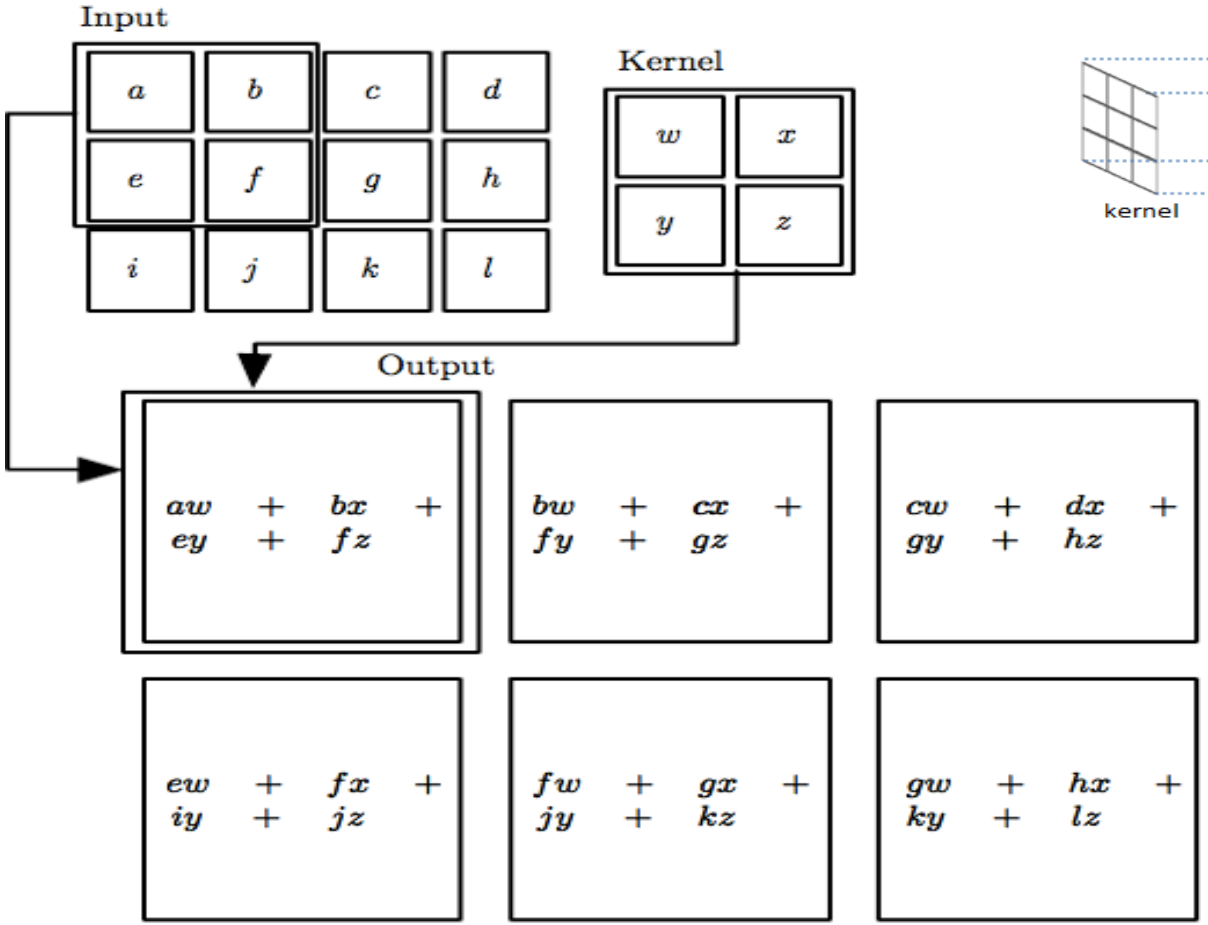
We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:

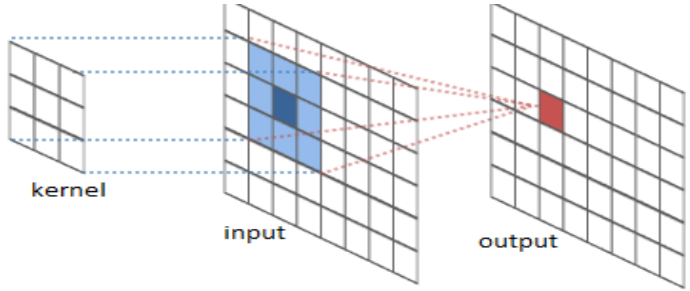


The convolution operation

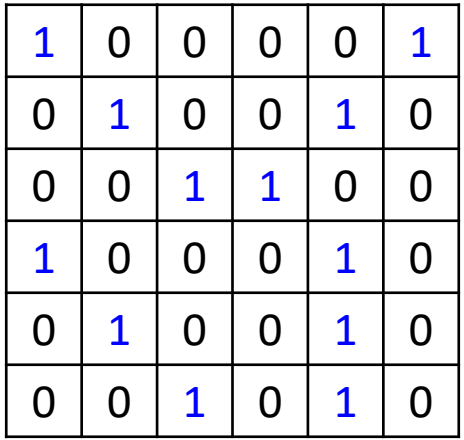


The convolution operation



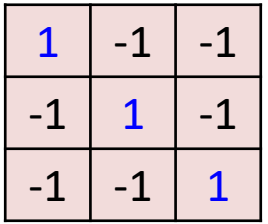


Convolution

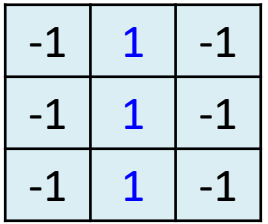


6 x 6 image

These are the network parameters to be learned.



Filter 1



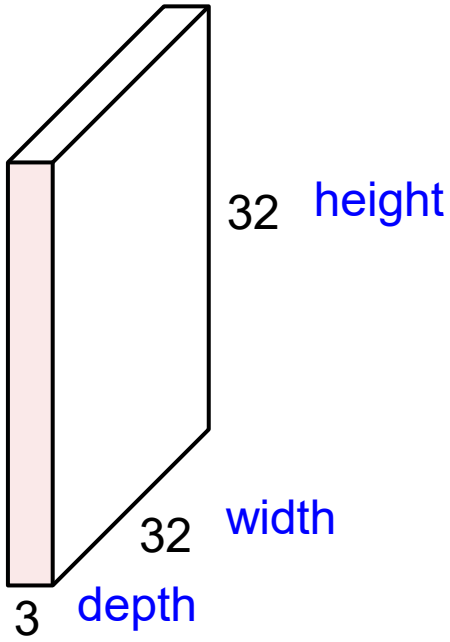
Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

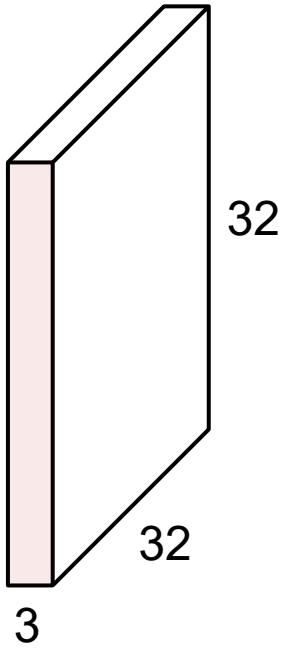
Convolution Layer

32x32x3 image

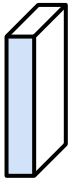


Convolution Layer

32x32x3 image



5x5x3 filter

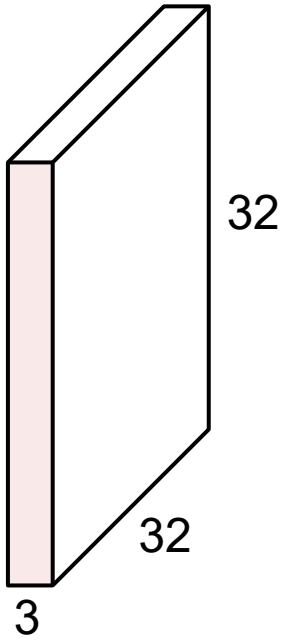


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

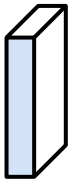
Convolution Layer

Filters always extend the full depth of the input volume

32x32x3 image

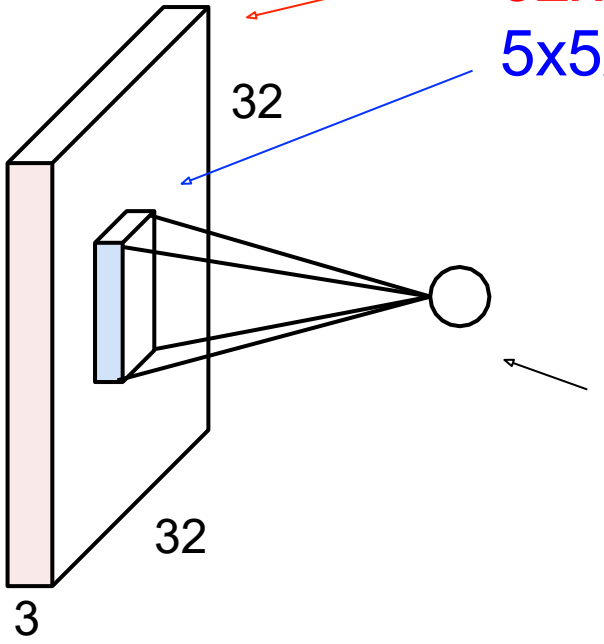


5x5x3 filter



Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

Convolution Layer



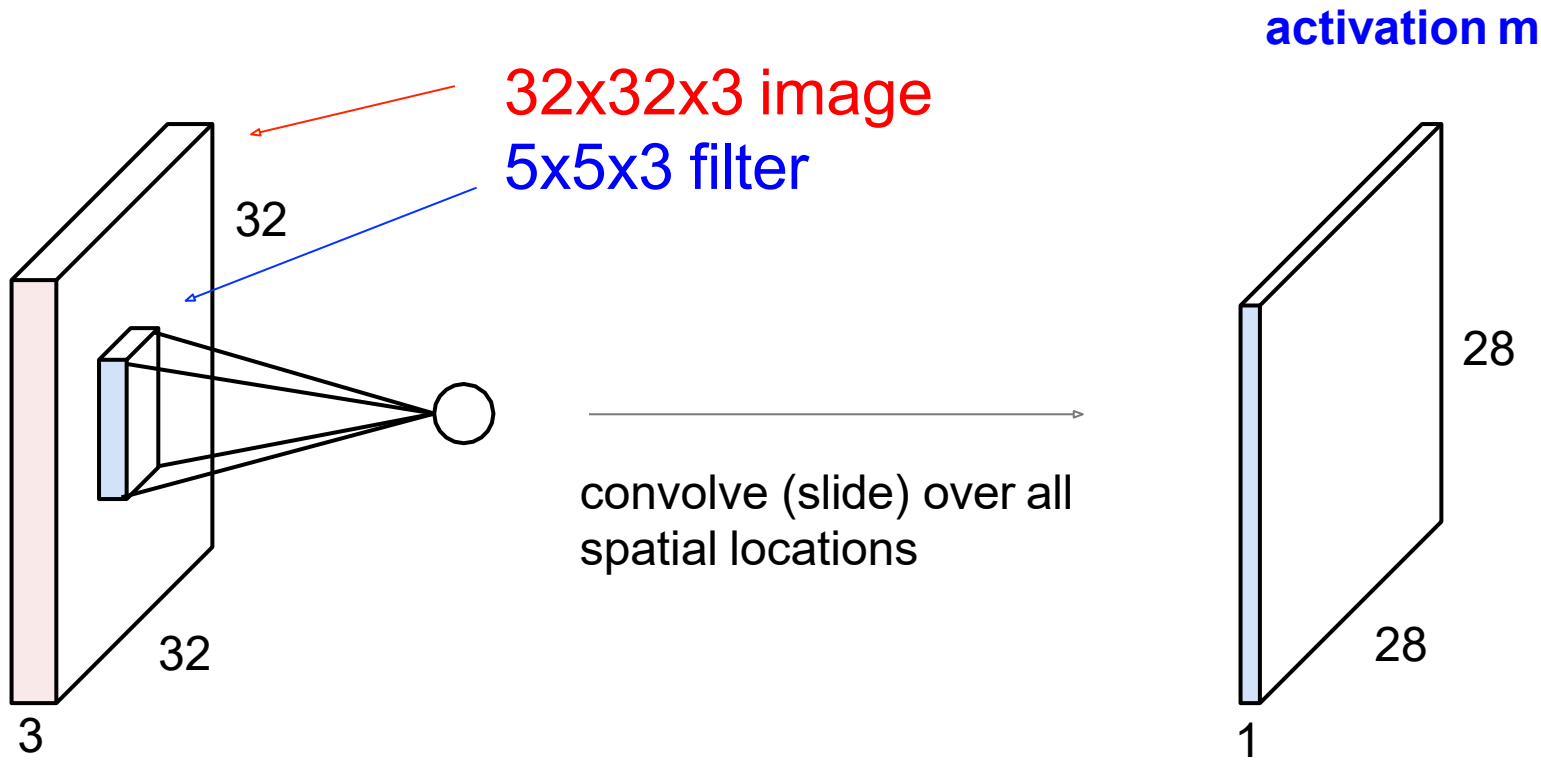
32x32x3 image

5x5x3 filter w

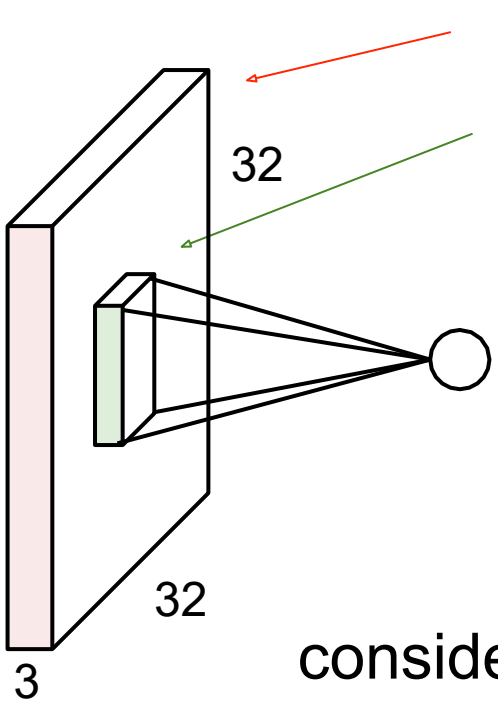
1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

Convolution Layer



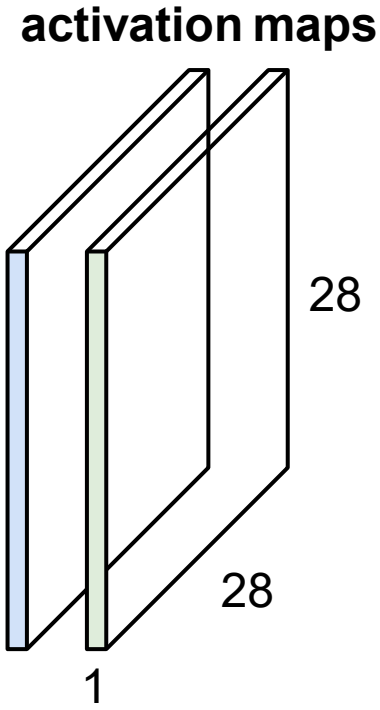
Convolution Layer



32x32x3 image

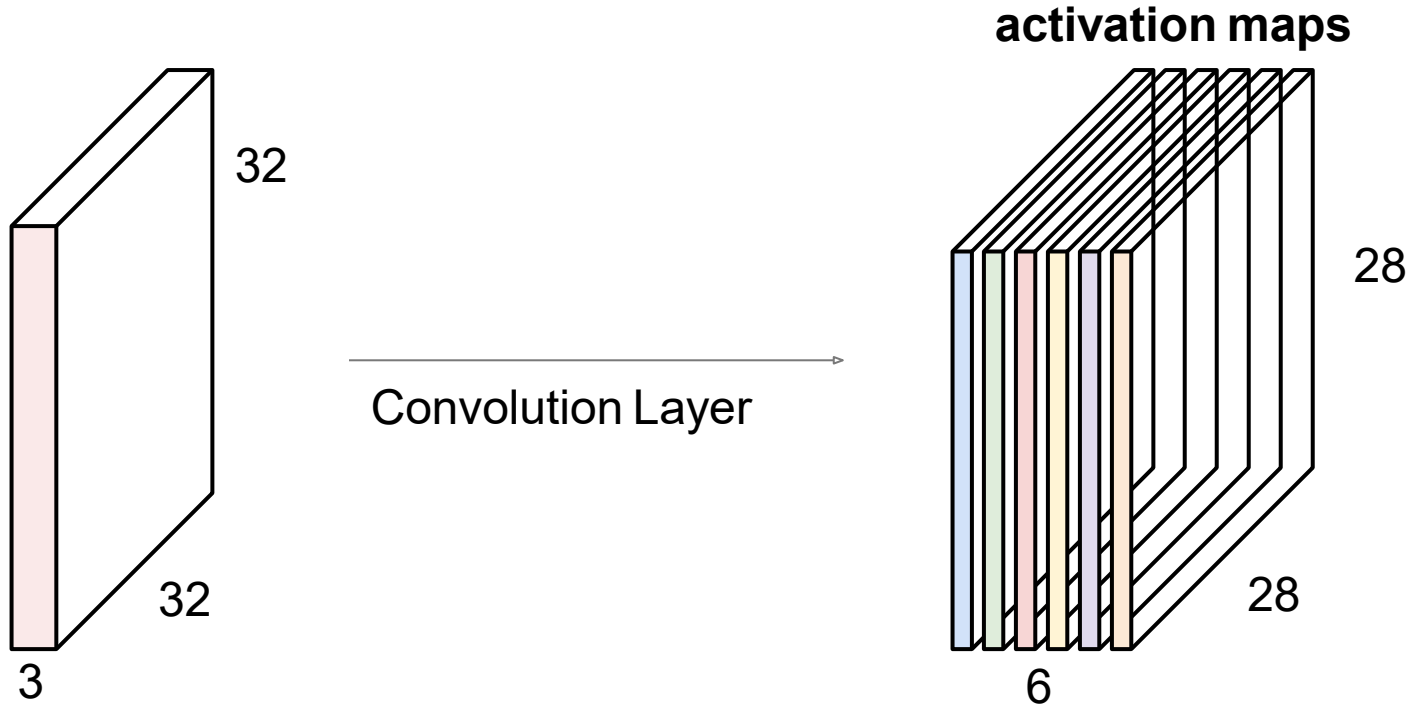
5x5x3 filter

convolve (slide) over all spatial locations



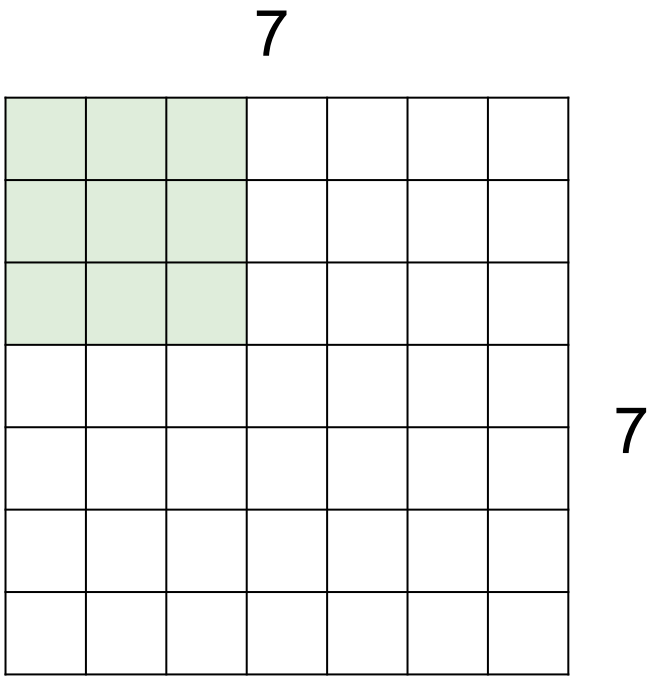
consider a second, green filter

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



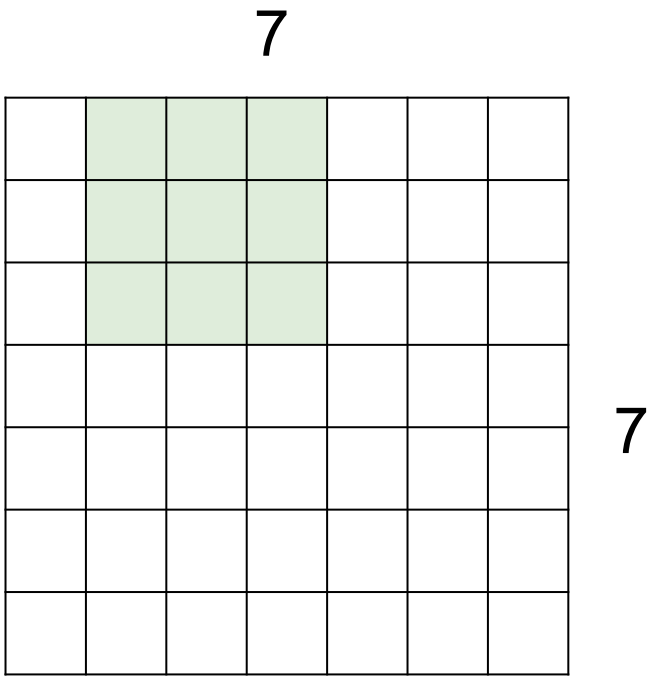
We stack these up to get a "new image" of size 28x28x6!

A closer look at spatial dimensions:



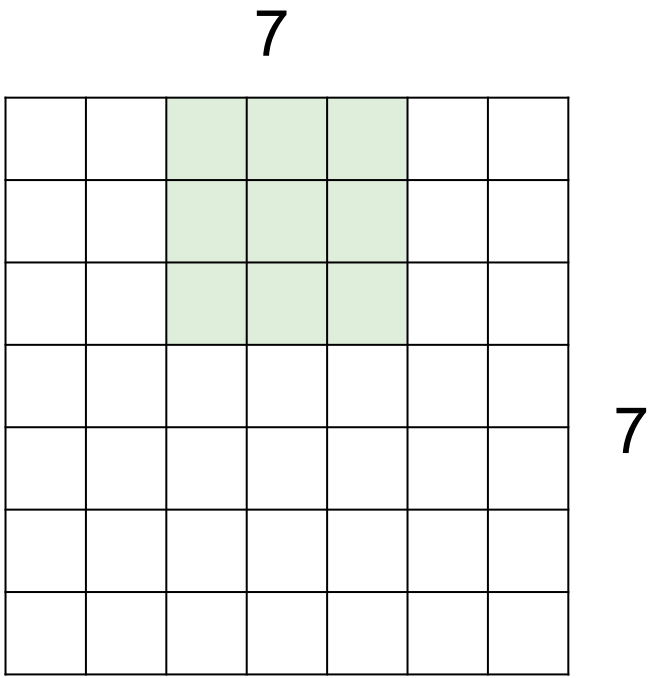
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



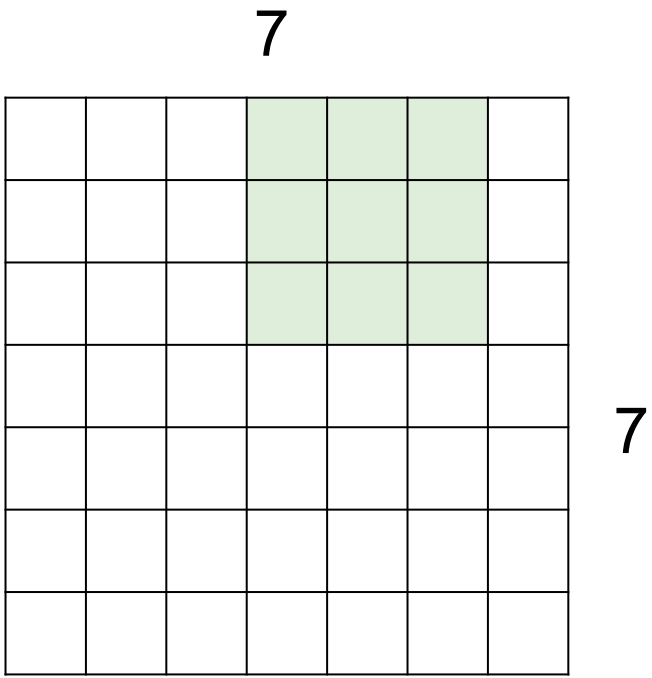
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



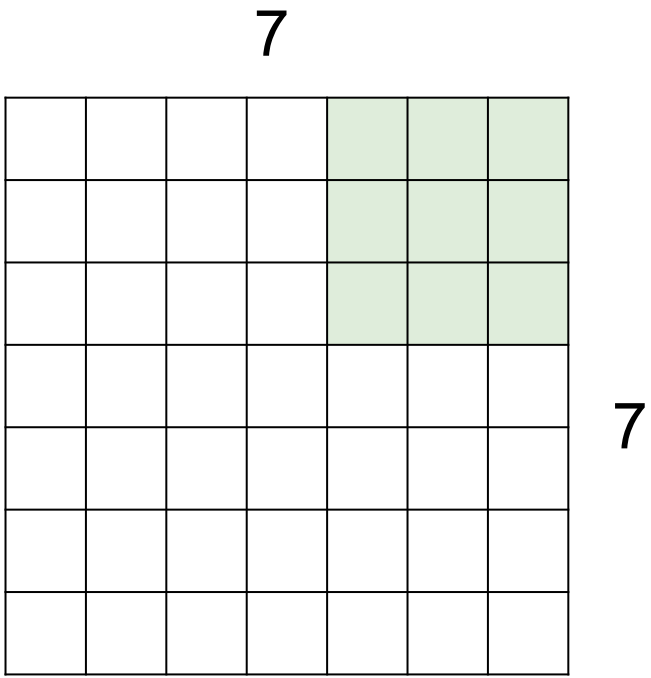
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

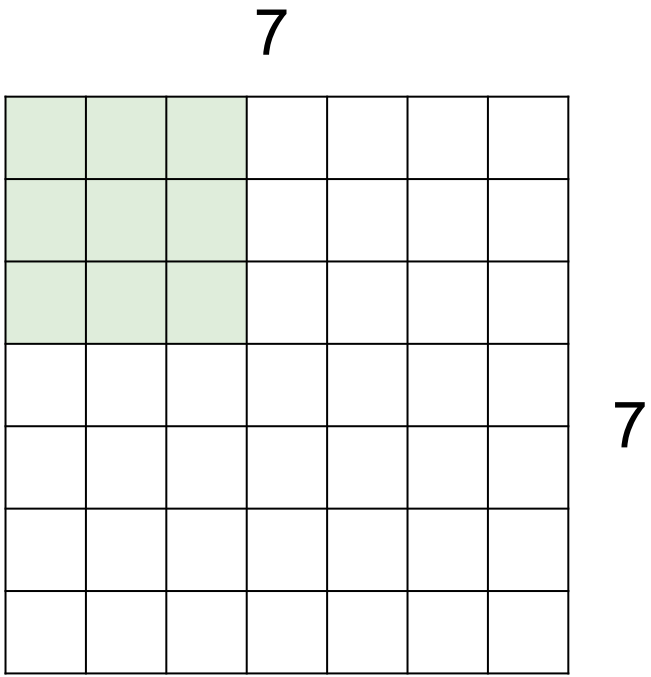
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

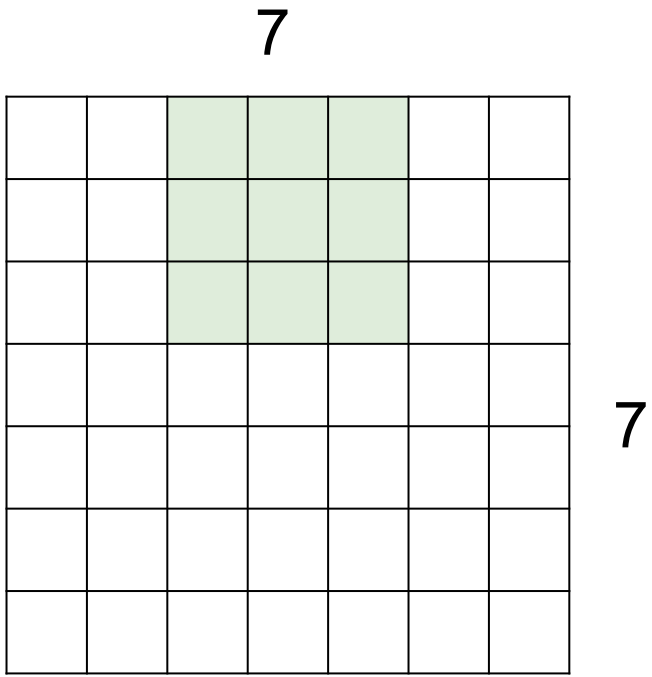
=> 5x5 output

A closer look at spatial dimensions:



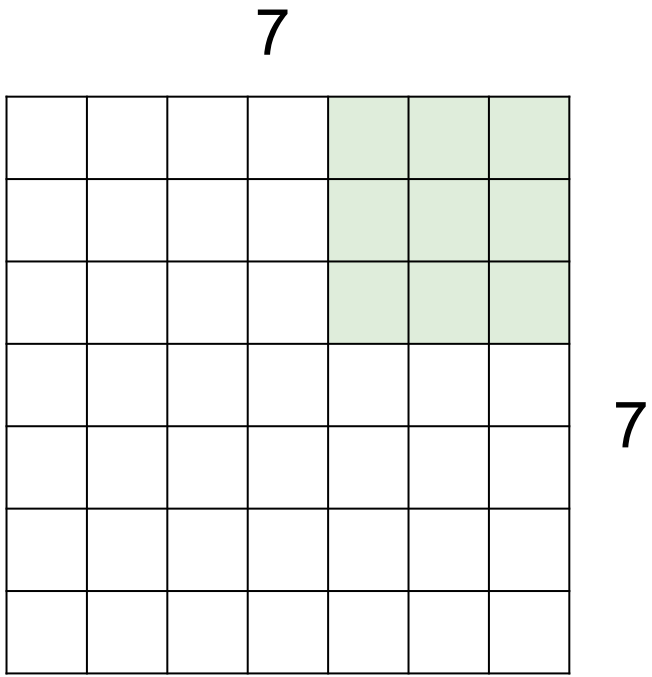
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



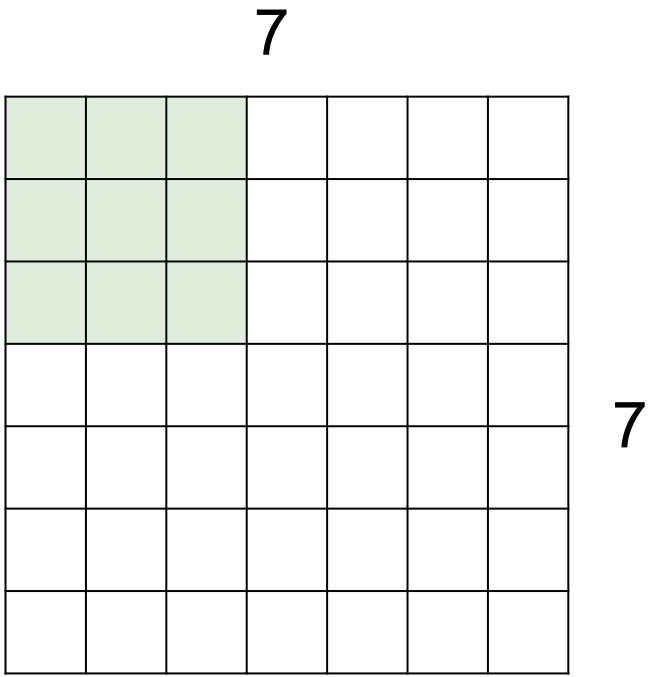
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



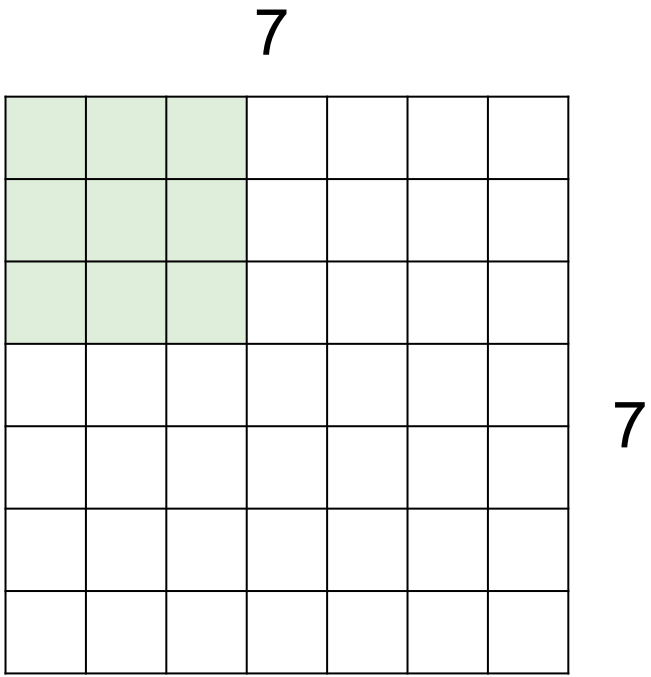
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

A closer look at spatial dimensions:



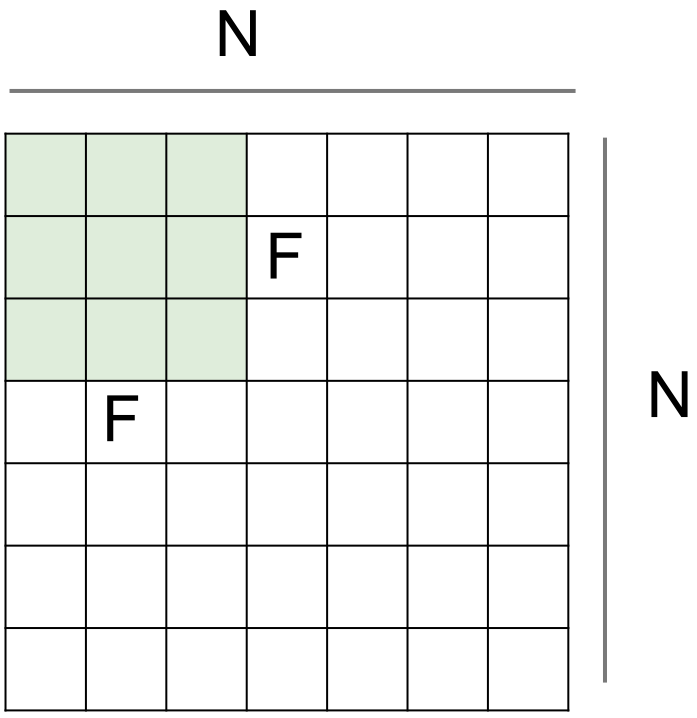
7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

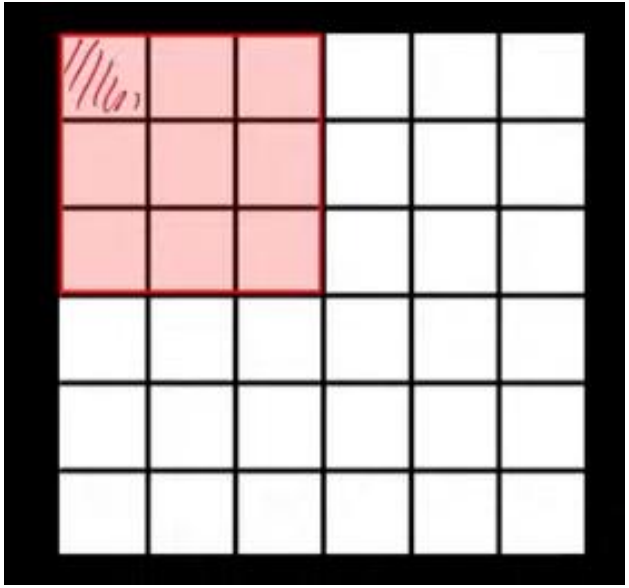
doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.

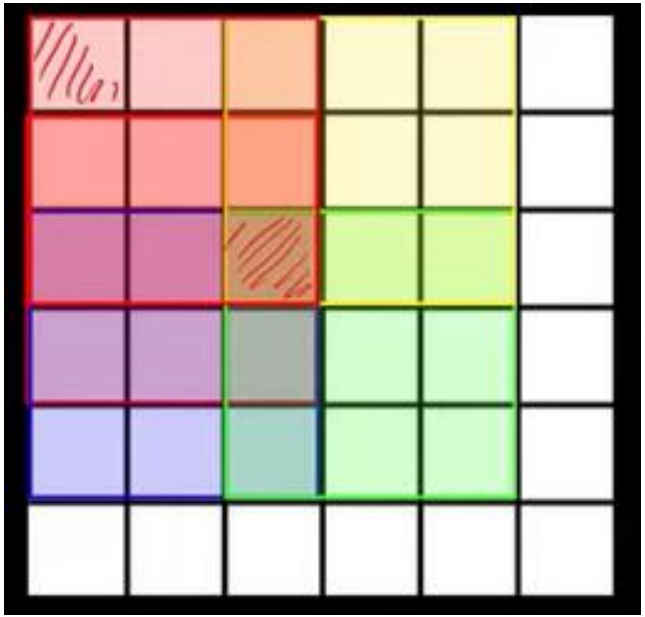


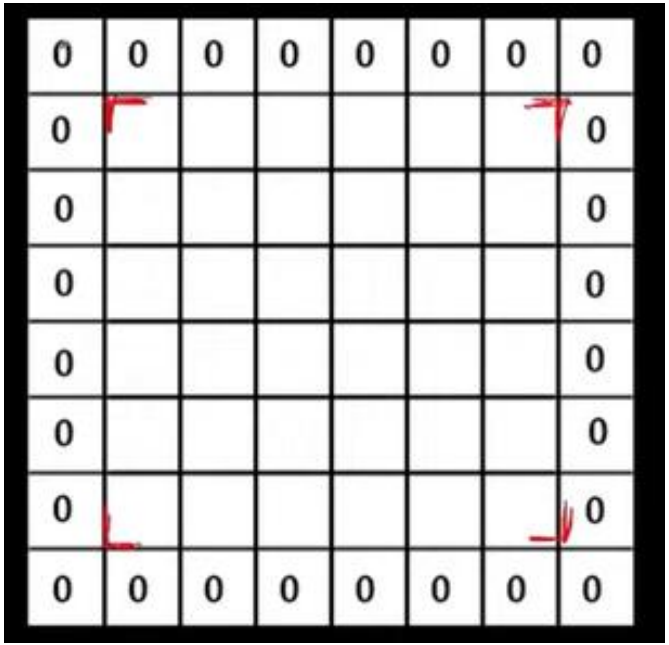
Output size:
(N - F) / stride + 1

e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33 :\

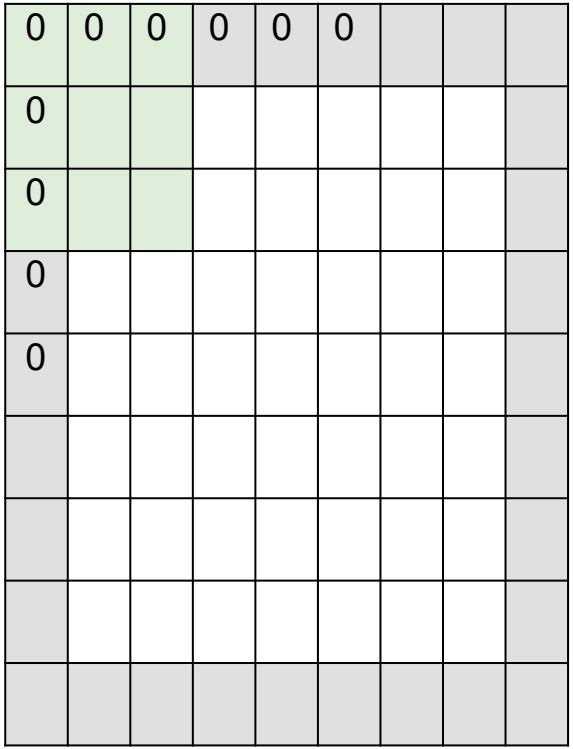
Zero-Padding






0 0 0 0 0 0 0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0 0 0 0 0 0 0

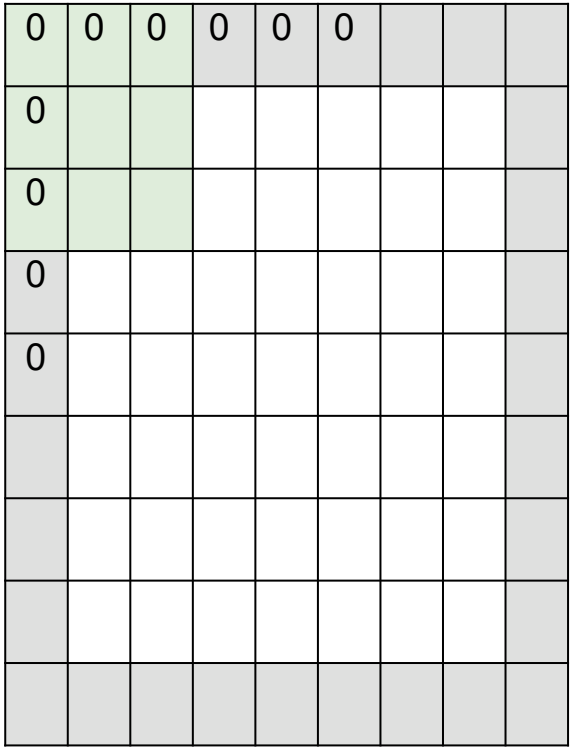
Zero-Padding: common to the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

(recall:)
(N - F) / stride + 1

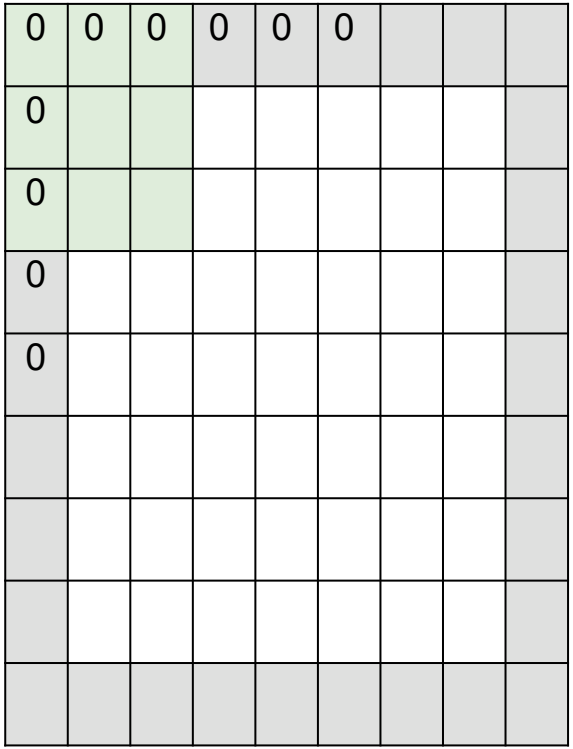
Zero-Padding: common to the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!

Zero-Padding: common to the border

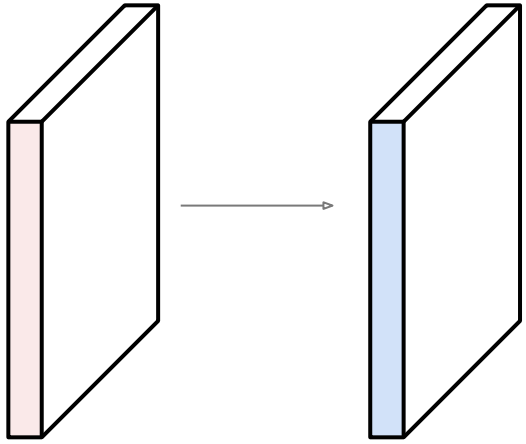


e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

(N – F+2P) / stride + 1

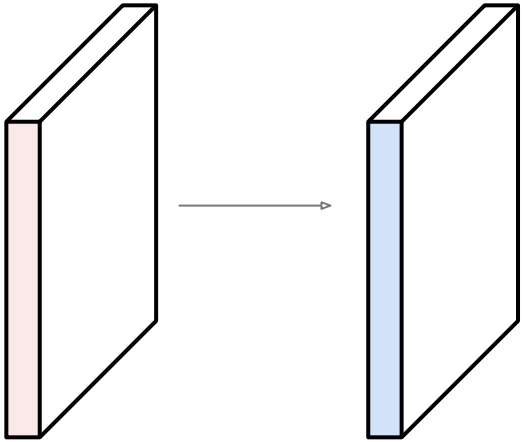
Examples time:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size: ?

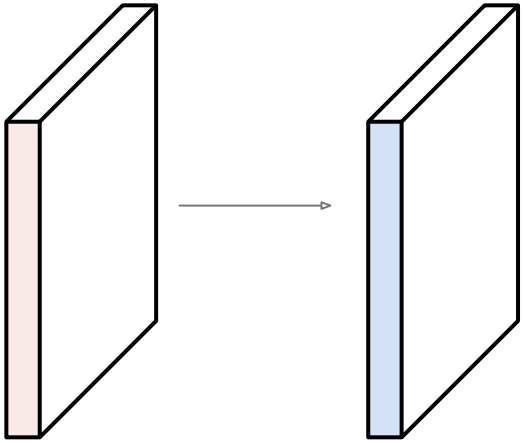
Examples time:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size:
(32+2*2-5)/1+1 = 32 spatially, so
32x32x10

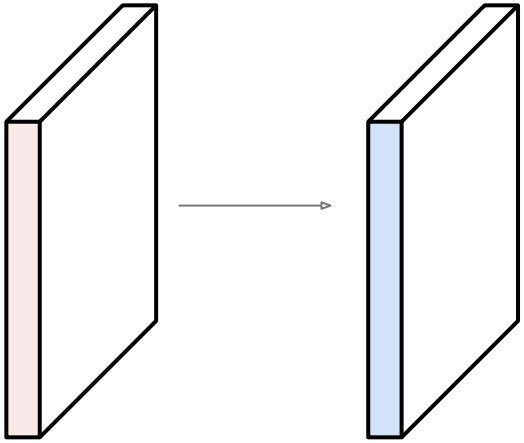
Examples time:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?

Examples time:

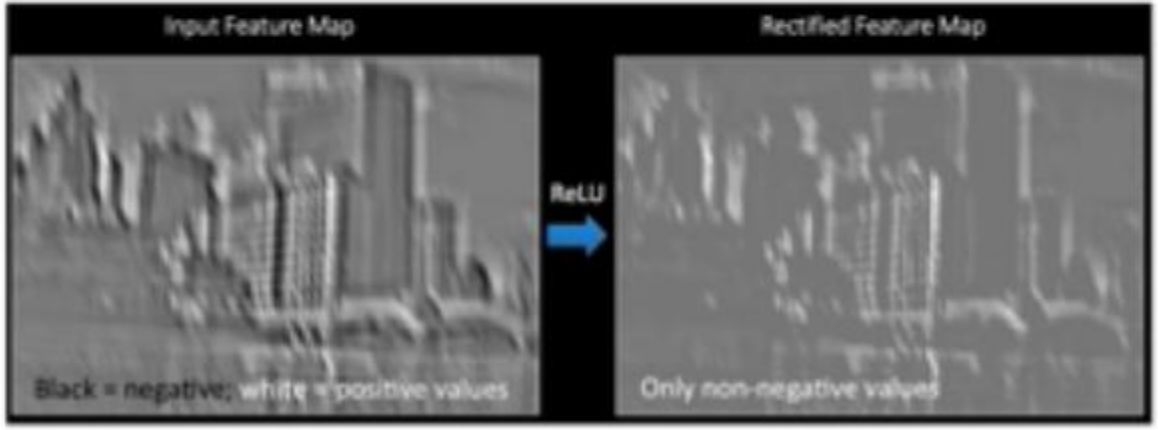


Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

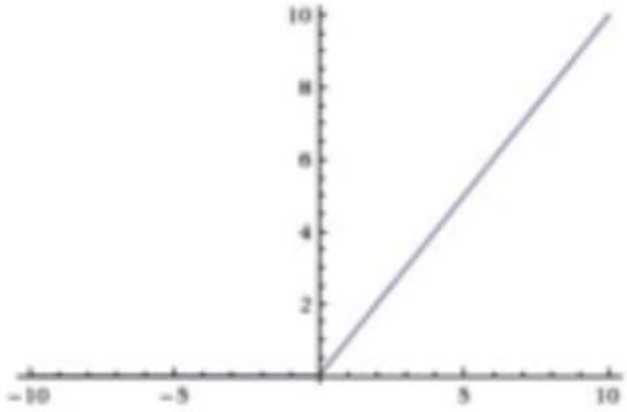
Number of parameters in this layer?
each filter has 5*5*3 + 1 = 76 params (+1 for bias)
=> 76*10 = **760**

Introducing Non-Linearity

- Apply after every convolution operation (i.e., after convolutional layers)
- ReLU: pixel-by-pixel operation that replaces all negative values by zero. **Non-linear operation!**



Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

`tf.keras.layers.ReLU`

Summary

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d-th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

Why Pooling

Subsampling pixels will not change the object

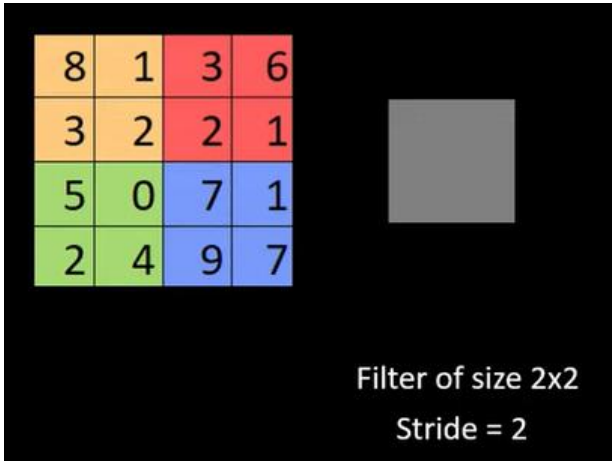
bird

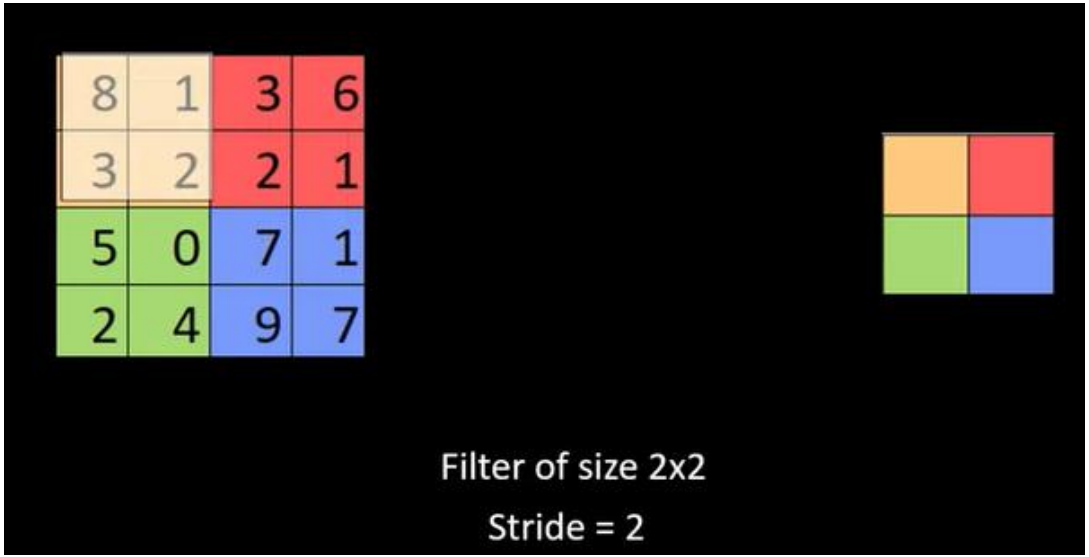
Subsampling

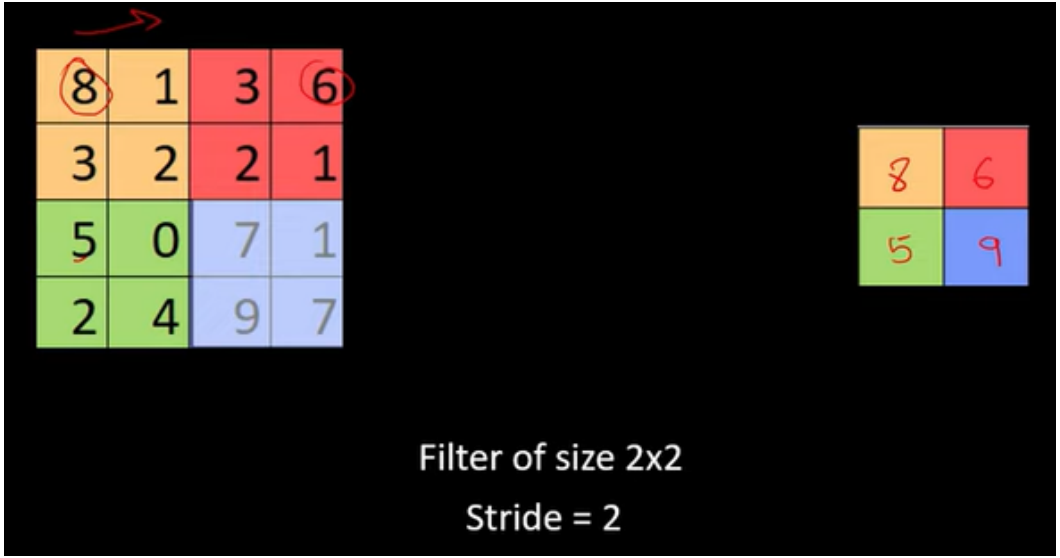
bird

We can subsample the pixels to make image smaller

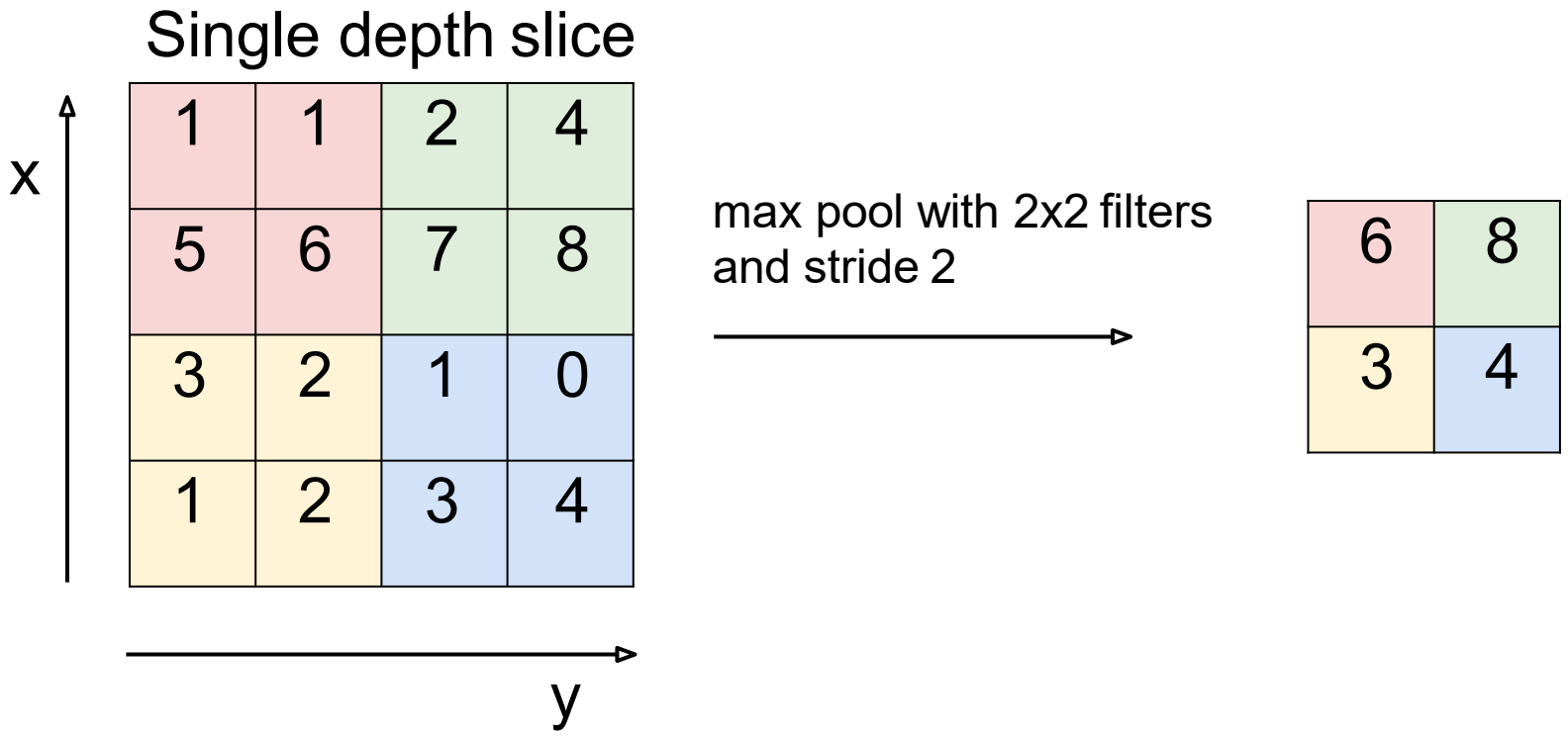
fewer parameters to **characterize** the **image**


8 1 3 6
3 2 2 1
5 0 7 1
2 4 9 7
Filter of size 2x2
Stride = 2


8 1 3 6
3 2 2 1
5 0 7 1
2 4 9 7
Filter of size 2x2
Stride = 2

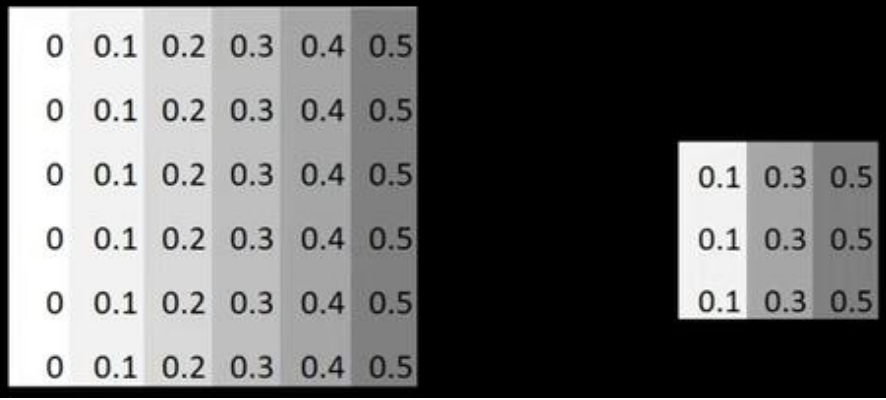

8 1 3 6
3 2 2 1
5 0 7 1
2 4 9 7
8 6
5 9
Filter of size 2x2
Stride = 2

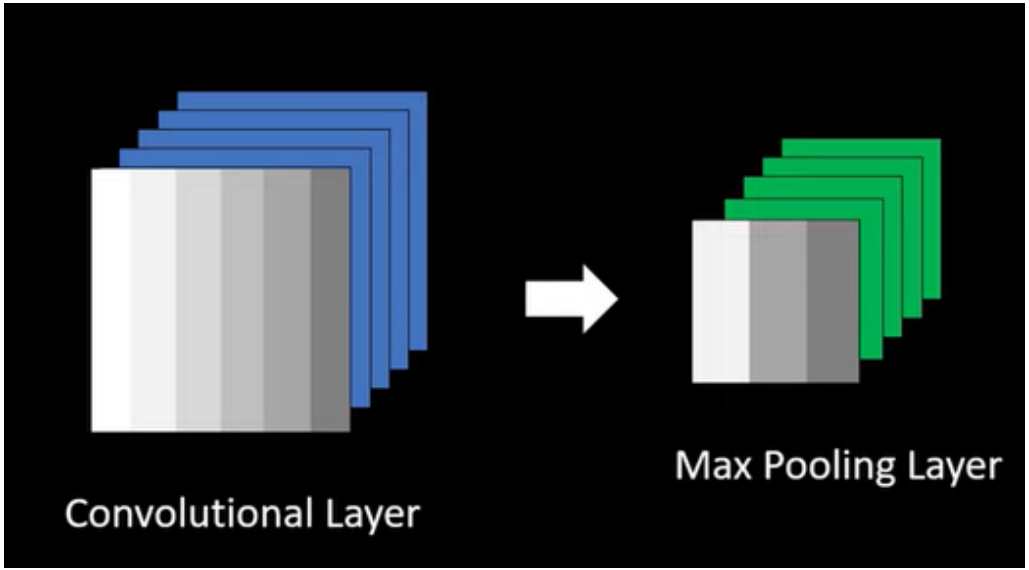
Max Pooling

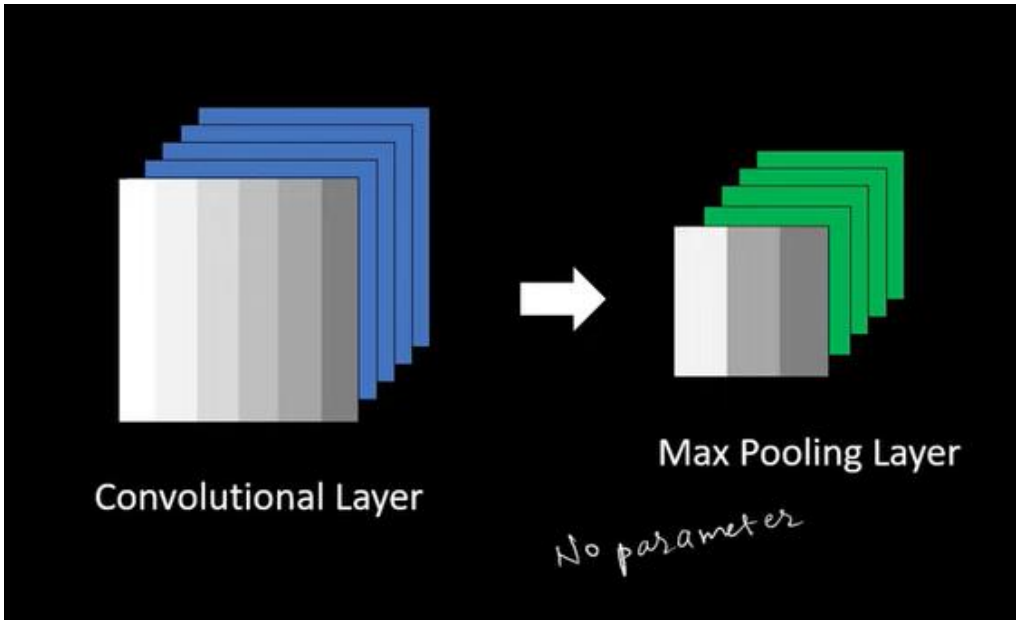

Single depth slice
x
y
1 1 2 4
5 6 7 8
3 2 1 0
1 2 3 4
max pool with 2x2 filters
and stride 2
6 8
3 4

Why do we need Max Pooling?

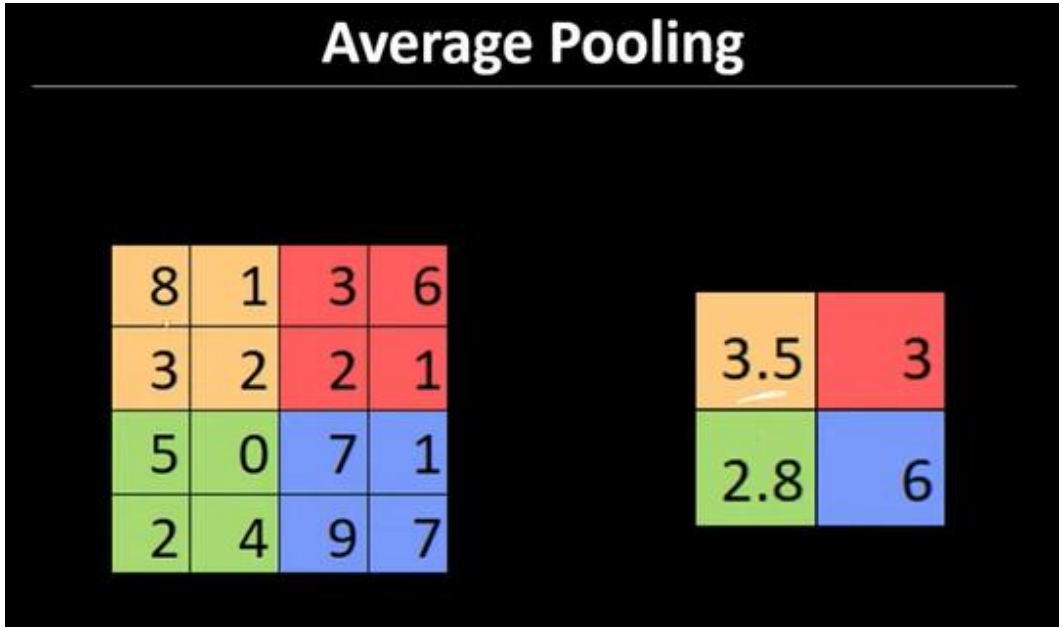
1. Reduce image size, thus reduce computational cost




Convolutional Layer
Max Pooling Layer


Convolutional Layer
Max Pooling Layer
No parameter

Average Pooling



- Pooling layers are generally used to reduce the size of the inputs and hence speed up the computation.

The hyperparameters for a pooling layer are:

- Filter size
- Stride
- Max or average pooling

Summary

Why do we need Max Pooling?

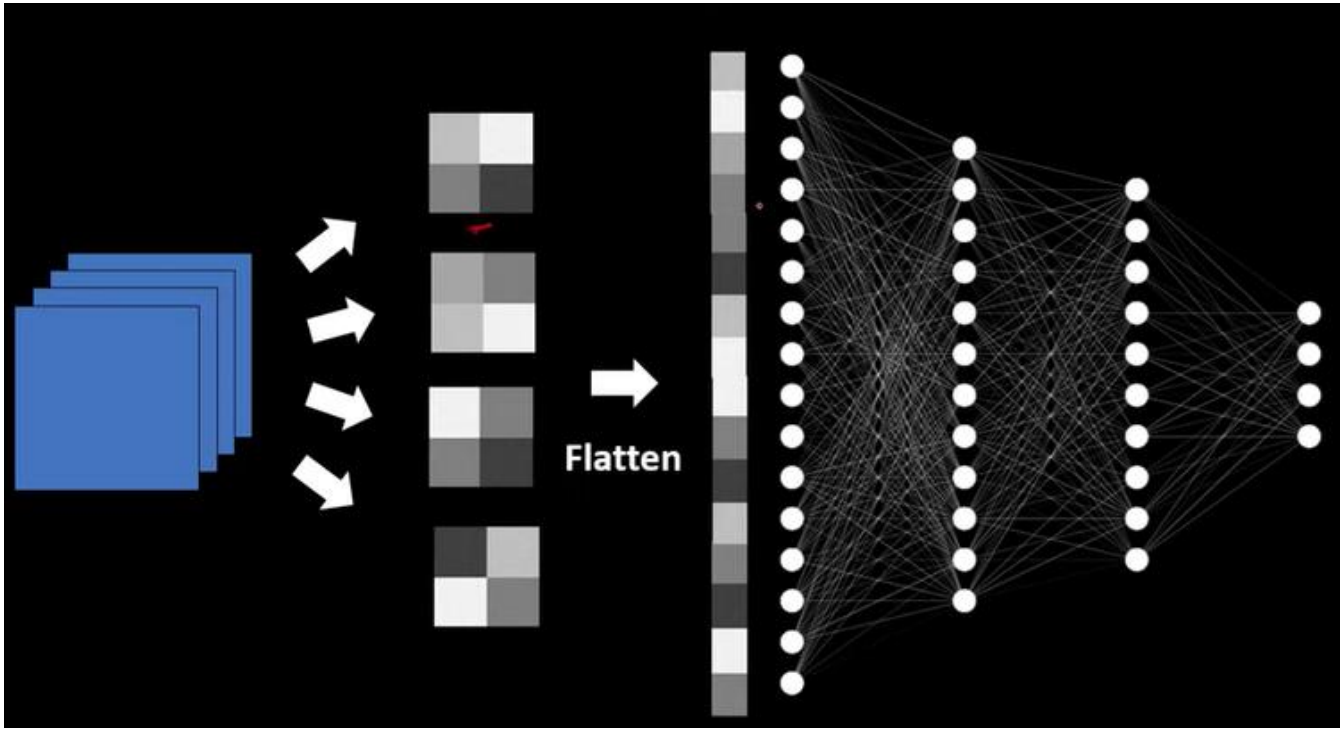
1. Reduce image size, thus reduce computational cost
2. Enhances the Features of the image

Where?

After Convolutional layer

Other points

1. No parameters involved, thus no training
2. Same number of channels in output as input


Flatten

Summary

- Fully connected layers are dense network of neurons.
- Applied after convolutional and max pooling layers.
- Classifies the output
- Associate features to a particular label

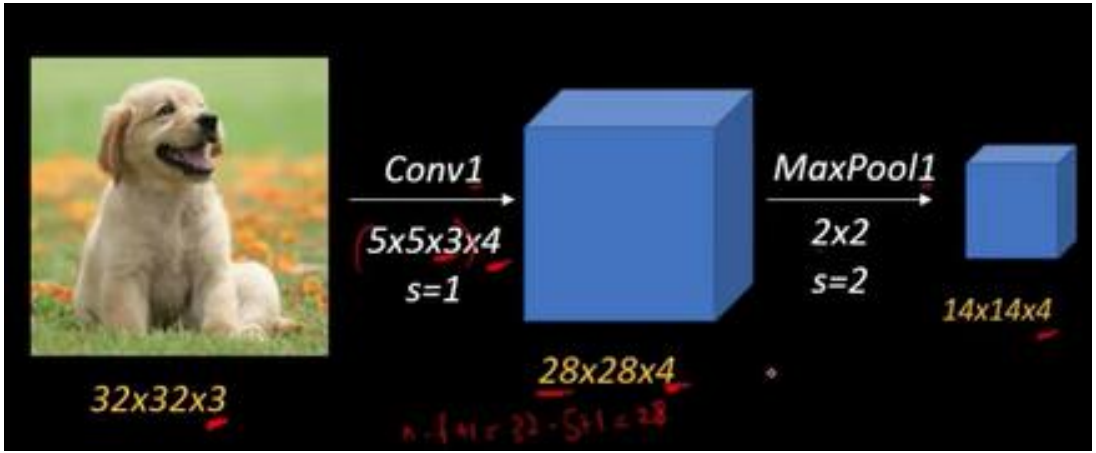
Convolutional Layer

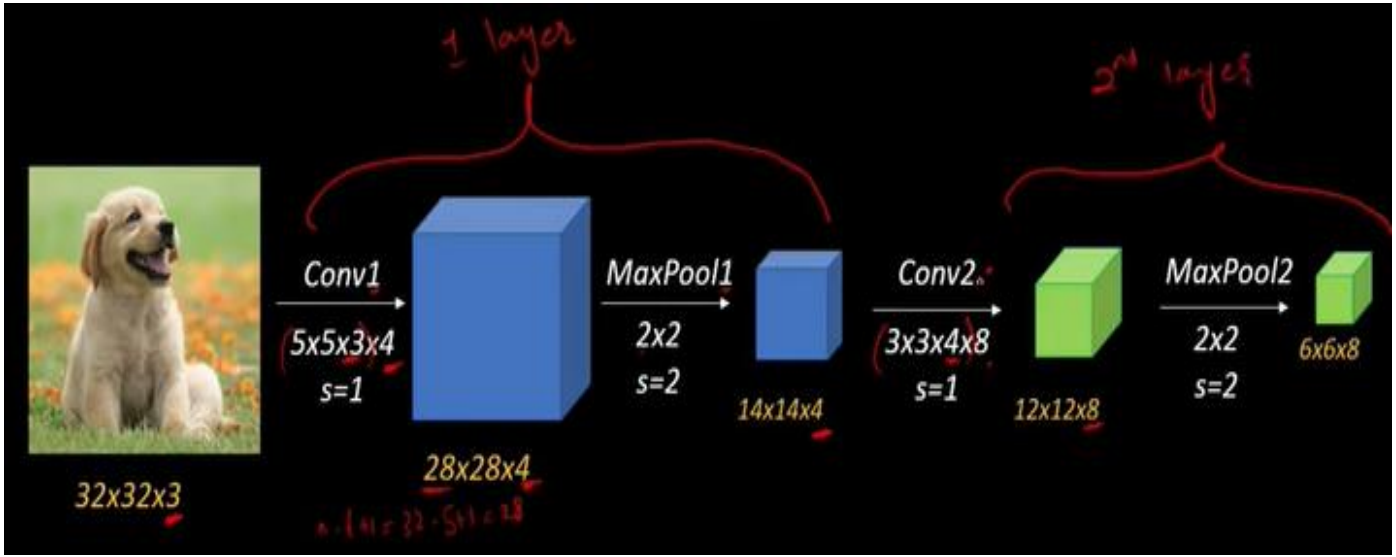
fxfx3

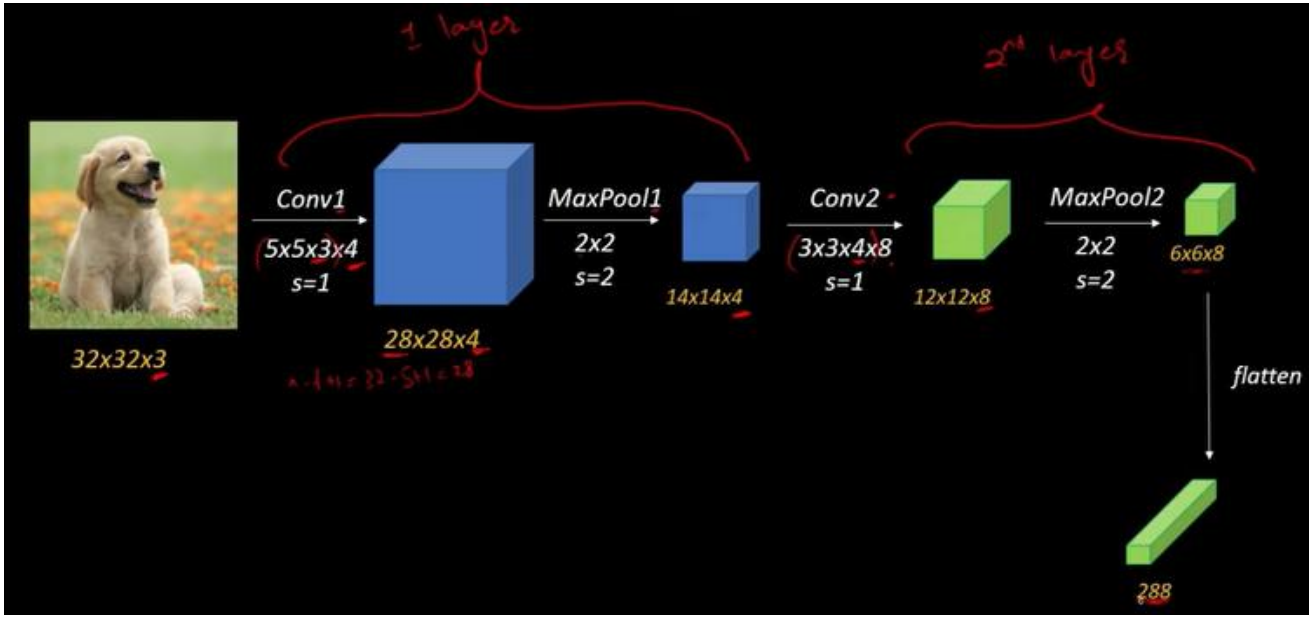
*

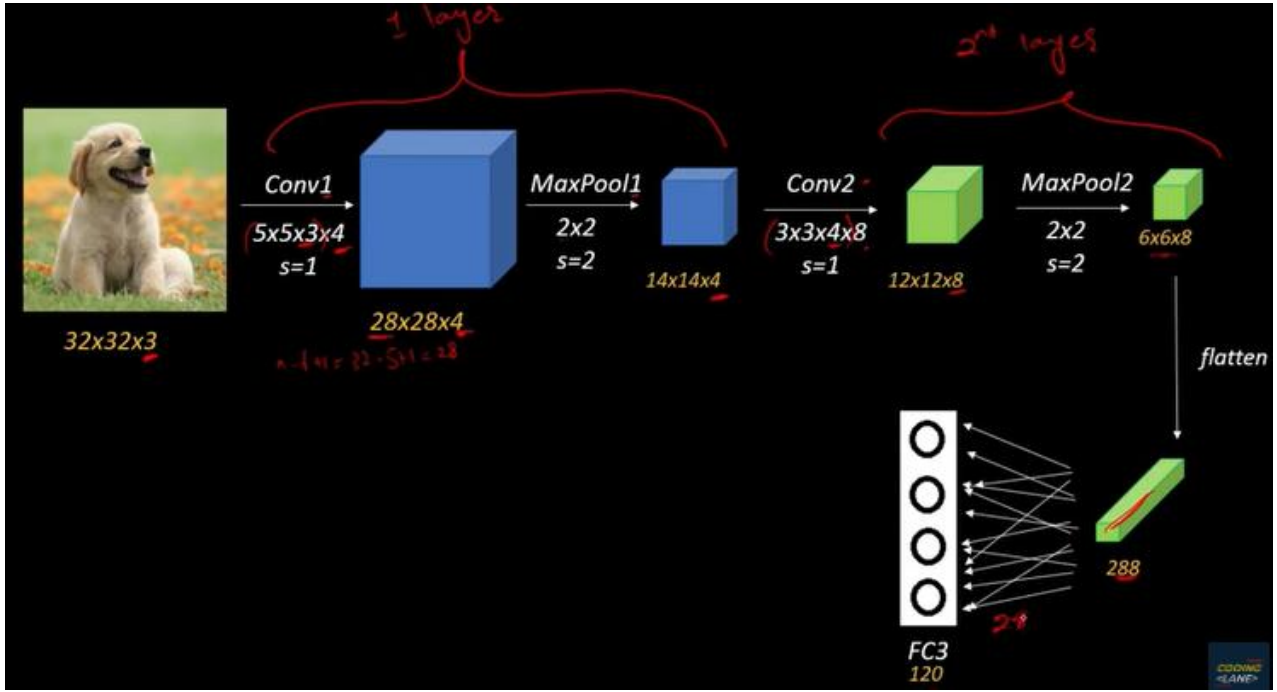
fxfx3

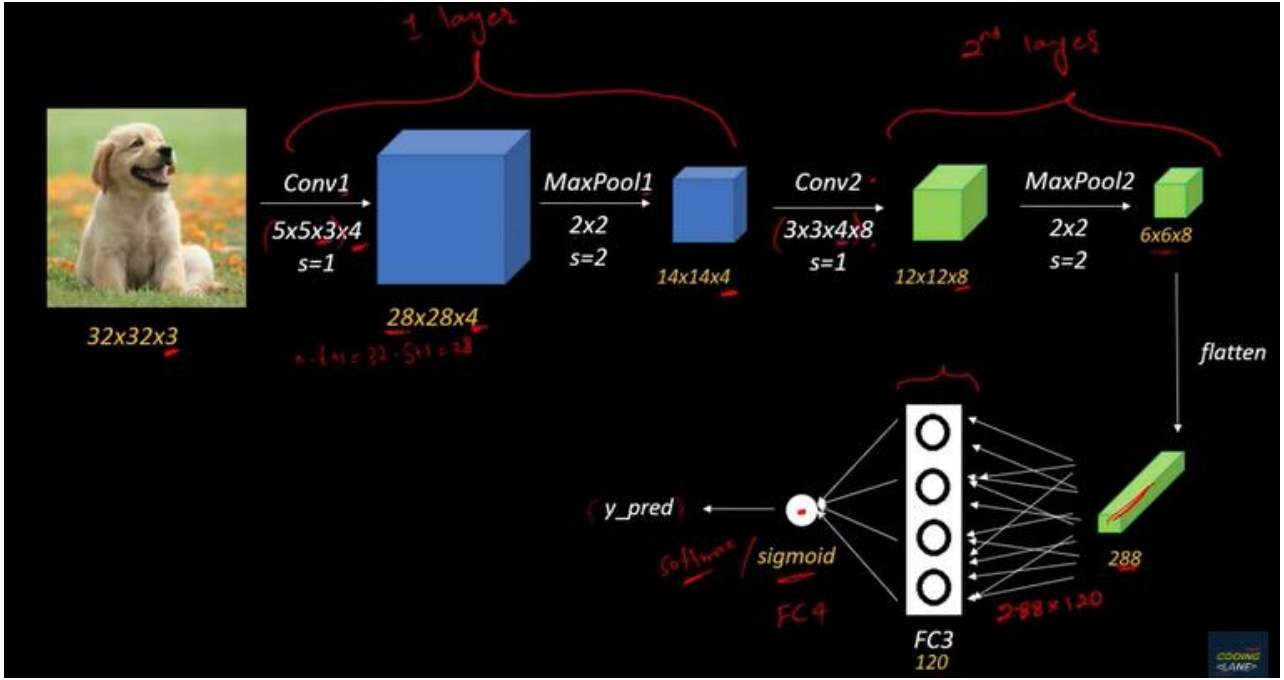
ReLU(+ b)

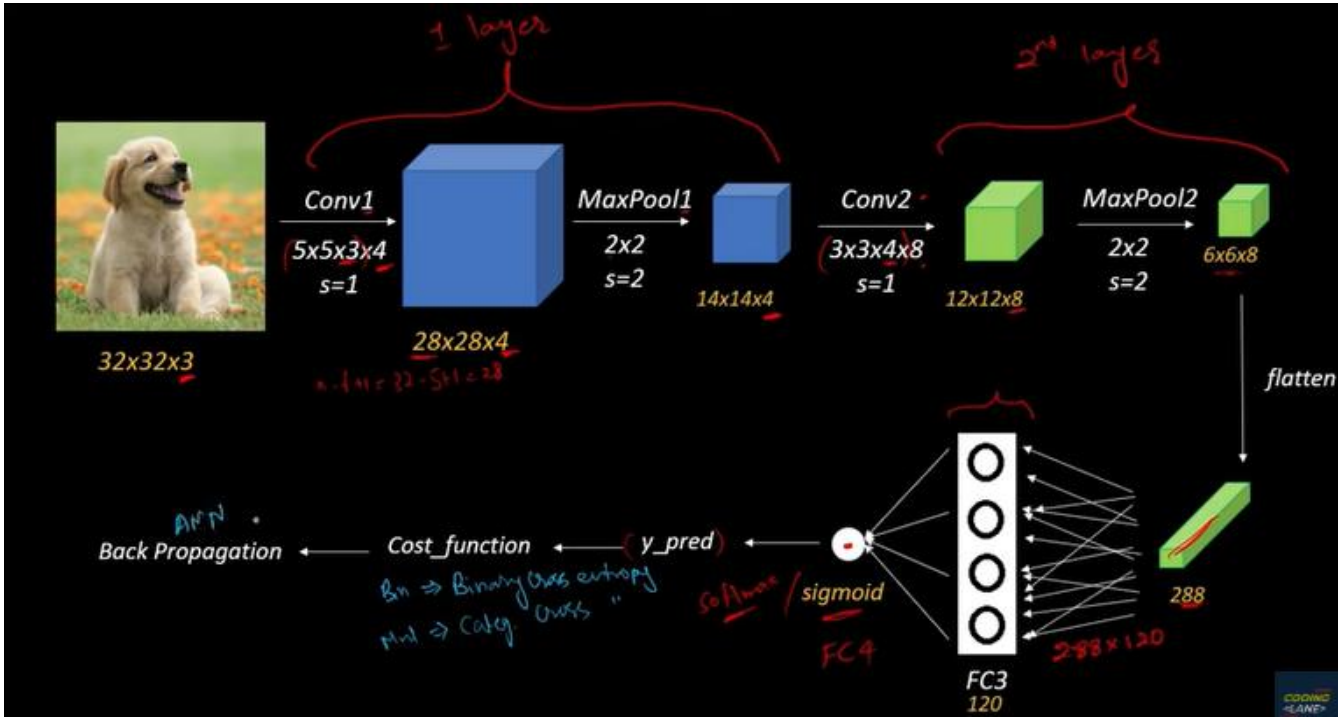

Conv1
5x5x3x4
s=1
MaxPool1
2x2
s=2
14x14x4
28x28x4
32x32x3

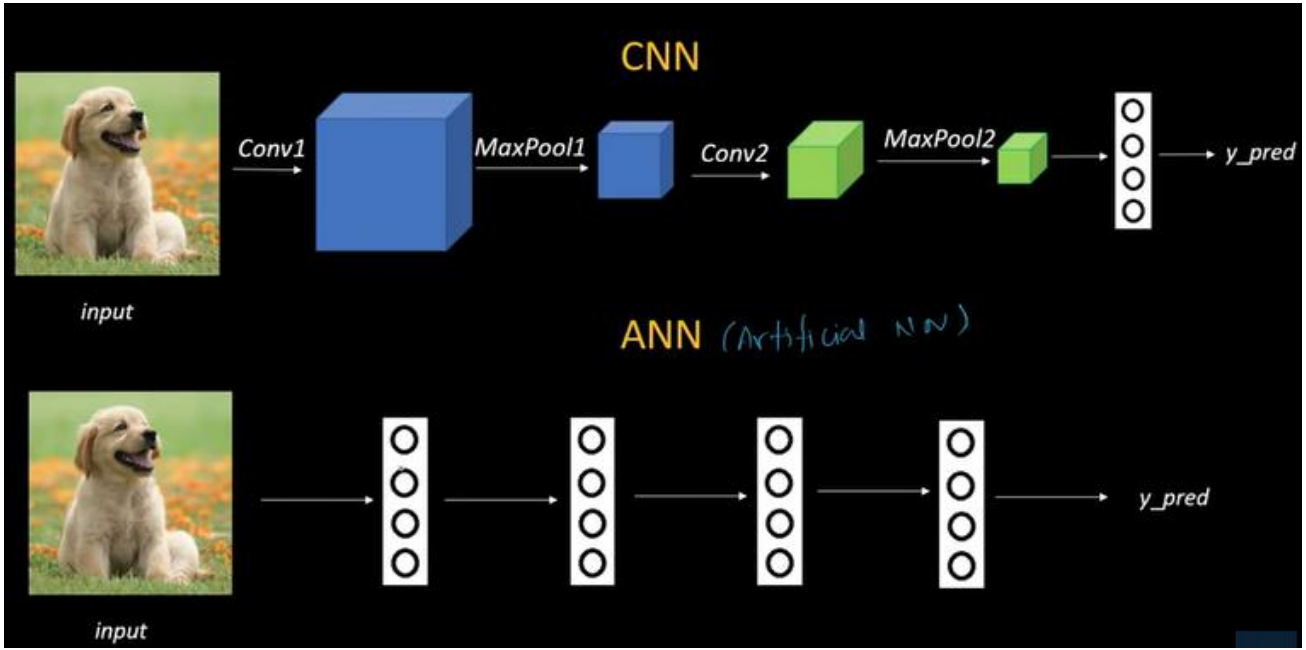

1 layer
2nd layer
Conv1
5x5x3x4
s=1
MaxPool1
2x2
s=2
Conv2
3x3x4x8
s=1
MaxPool2
2x2
s=2
32x32x3
28x28x4
14x14x4
12x12x8
6x6x8

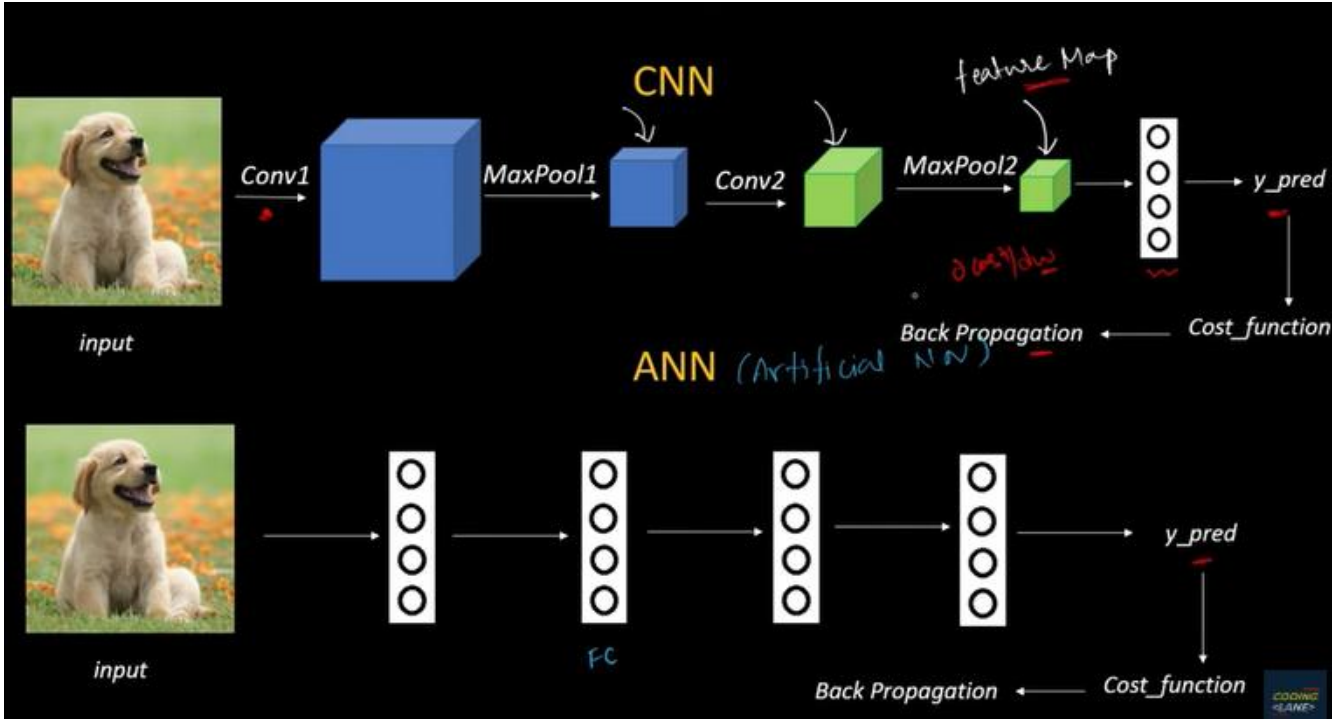

1 layer
2nd layer
Conv1
5x5x3x4
s=1
MaxPool1
2x2
s=2
Conv2
3x3x4x8
s=1
MaxPool2
2x2
s=2
32x32x3
28x28x4
14x14x4
12x12x8
6x6x8
flatten
288


1 layer
2nd layer
Conv1
5x5x3x4
s=1
MaxPool1
2x2
s=2
Conv2
3x3x4x8
s=1
MaxPool2
2x2
s=2
32x32x3
28x28x4
14x14x4
12x12x8
6x6x8
flatten
288
FC3
120


1 layer
2nd layer
32x32x3
Conv1
5x5x3x4
s=1
28x28x4
MaxPool1
2x2
s=2
14x14x4
Conv2
3x3x4x8
s=1
12x12x8
MaxPool2
2x2
s=2
6x6x8
flatten
288
FC3
120
y_pred
sigmoid
FC4


1 layer
2nd layer
Conv1
5x5x3x4
s=1
MaxPool1
2x2
s=2
Conv2
3x3x4x8
s=1
MaxPool2
2x2
s=2
6x6x8
32x32x3
28x28x4
14x14x4
12x12x8
flatten
288
FC3
120
FC4
sigmoid
y_pred
Cost_function
Back Propagation
ANN
288x120
CODING LANE


CNN
Conv1
MaxPool1
Conv2
MaxPool2
y_pred
input
ANN (Artificial NN)
y_pred
input


CNN
feature Map
Conv1
MaxPool1
Conv2
MaxPool2
y_pred
input
Back Propagation
Cost_function
ANN (Artificial NN)
y_pred
input
FC
Back Propagation
Cost_function

Questions!!